In [1]:
```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
```

In [3]:
```python
retail = pd.read_csv('Retail_Data_Transactions.csv')
```

In [4]:
```python
retail
```

Out[4]:

	customer_id	trans_date	tran_amount
0	CS5295	11-Feb-13	35
1	CS4768	15-Mar-15	39
2	CS2122	26-Feb-13	52
3	CS1217	16-Nov-11	99
4	CS1850	20-Nov-13	78
...	...	...	...
124995	CS8433	26-Jun-11	64
124996	CS7232	19-Aug-14	38
124997	CS8731	28-Nov-14	42
124998	CS8133	14-Dec-13	13
124999	CS7996	13-Dec-14	36

125000 rows × 3 columns

In [5]:
```python
retail.isnull()
```

Out[5]:

	customer_id	trans_date	tran_amount
0	False	False	False
1	False	False	False
2	False	False	False
3	False	False	False
4	False	False	False
...	...	...	...
124995	False	False	False
124996	False	False	False
124997	False	False	False
124998	False	False	False
124999	False	False	False

125000 rows × 3 columns

In [7]:
```python
retail['trans_date'] = pd.to_datetime(retail['trans_date'])
```

```
C:\Users\Manisha\AppData\Local\Temp\ipykernel_10324\962200029.py:1: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consist
ent and as-expected, please specify a format.
  retail['trans_date'] = pd.to_datetime(retail['trans_date'])
```

In [8]:
```python
retail
```

Out[8]:

	customer_id	trans_date	tran_amount
0	CS5295	2013-02-11	35
1	CS4768	2015-03-15	39
2	CS2122	2013-02-26	52
3	CS1217	2011-11-16	99
4	CS1850	2013-11-20	78
...	...	...	...
124995	CS8433	2011-06-26	64
124996	CS7232	2014-08-19	38
124997	CS8731	2014-11-28	42
124998	CS8133	2013-12-14	13
124999	CS7996	2014-12-13	36

125000 rows × 3 columns

In [9]:
```python
response = pd.read_csv('Retail_Data_Response.csv')
```

In [10]:
```python
response
```

Out[10]:

	customer_id	response
0	CS1112	0
1	CS1113	0
2	CS1114	1
3	CS1115	1
4	CS1116	1
...	...	...
6879	CS8996	0
6880	CS8997	0
6881	CS8998	0
6882	CS8999	0
6883	CS9000	0

6884 rows × 2 columns

In [11]:
```python
df = pd.merge(retail, response, on = 'customer_id', how = 'left')
```

In [12]:
```python
df
```

Out[12]:

	customer_id	trans_date	tran_amount	response
0	CS5295	2013-02-11	35	1.0
1	CS4768	2015-03-15	39	1.0
2	CS2122	2013-02-26	52	0.0
3	CS1217	2011-11-16	99	0.0
4	CS1850	2013-11-20	78	0.0
...	...	...	...	...
124995	CS8433	2011-06-26	64	0.0
124996	CS7232	2014-08-19	38	0.0
124997	CS8731	2014-11-28	42	0.0
124998	CS8133	2013-12-14	13	0.0
124999	CS7996	2014-12-13	36	0.0

125000 rows × 4 columns

In [16]:
```python
df.dtypes
```

Out[16]:
```
customer_id            object
trans_date     datetime64[ns]
tran_amount             int64
response              float64
dtype: object
```

In [17]:
```python
df.describe()
```

Out[17]:

	trans_date	tran_amount	response
count	125000	125000.000000	124969.000000
mean	2013-04-13 12:01:16.031999744	64.991912	0.110763
min	2011-05-16 00:00:00	10.000000	0.000000
25%	2012-04-29 00:00:00	47.000000	0.000000
50%	2013-04-14 00:00:00	65.000000	0.000000
75%	2014-03-28 00:00:00	83.000000	0.000000
max	2015-03-16 00:00:00	105.000000	1.000000
std	NaN	22.860006	0.313840

In [19]:
```python
df.index
```

Out[19]:
```
RangeIndex(start=0, stop=125000, step=1)
```

In [47]:
```python
filtered = df[df['tran_amount'] >=100]
```

In [48]:
```python
filtered
```

Out[48]:

	customer_id	trans_date	tran_amount	response
18	CS3904	2014-07-20	103	0.0
29	CS2509	2011-12-05	105	0.0
44	CS2845	2013-03-08	105	0.0
66	CS2051	2014-12-09	101	0.0
74	CS1939	2011-11-25	101	0.0
...	...	...	...	...
99943	CS2947	2013-09-18	103	0.0
99948	CS2951	2012-04-27	101	1.0
99956	CS3365	2011-06-02	103	0.0
99963	CS3327	2011-07-27	105	0.0
99996	CS5050	2012-12-03	104	0.0

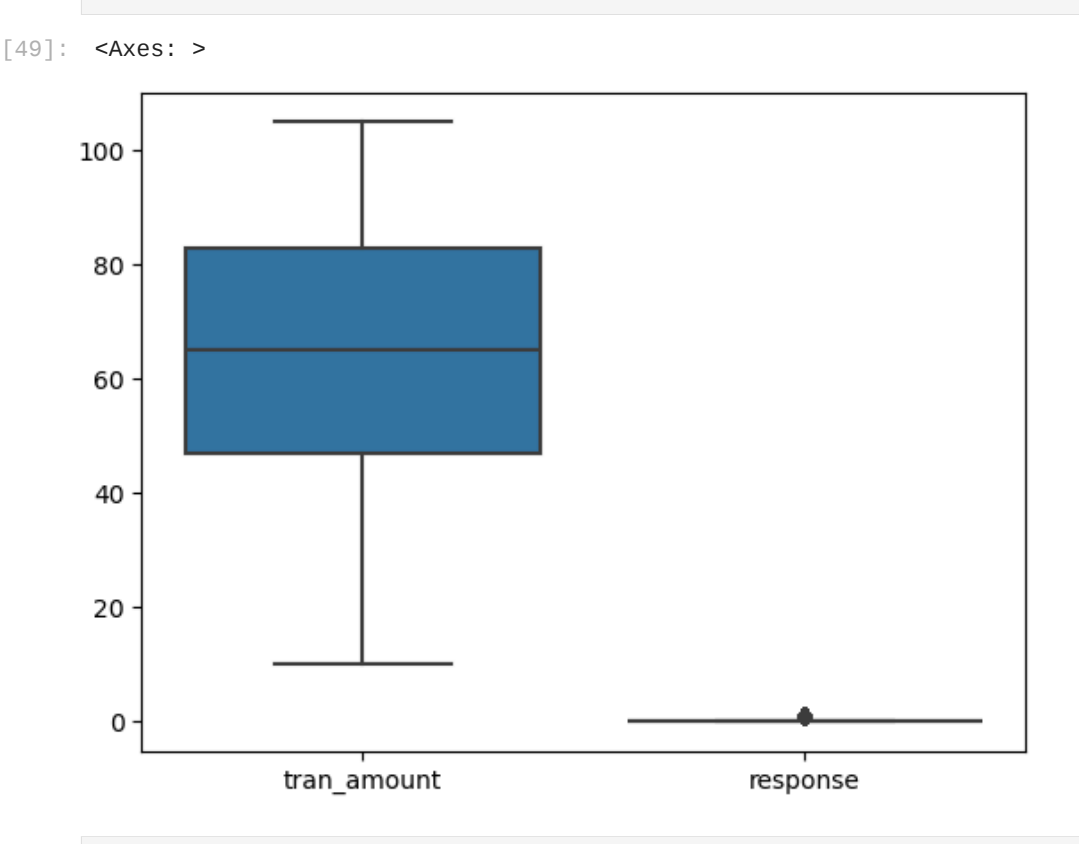
8466 rows × 4 columns

In [26]:
```python
df.duplicated()
```

Out[26]:
```
0         False
1         False
2         False
3         False
4         False
          ...  
124995    False
124996    False
124997    False
124998    False
124999    False
Length: 125000, dtype: bool
```

In [49]:
```python
sns.boxplot(df)
```

Out[49]:
```
<Axes: >
```



In [53]:
```python
df['tran_amount'].mean()
```

Out[53]:
```
64.991912
```

In [54]:
```python
df['tran_amount'].std()
```

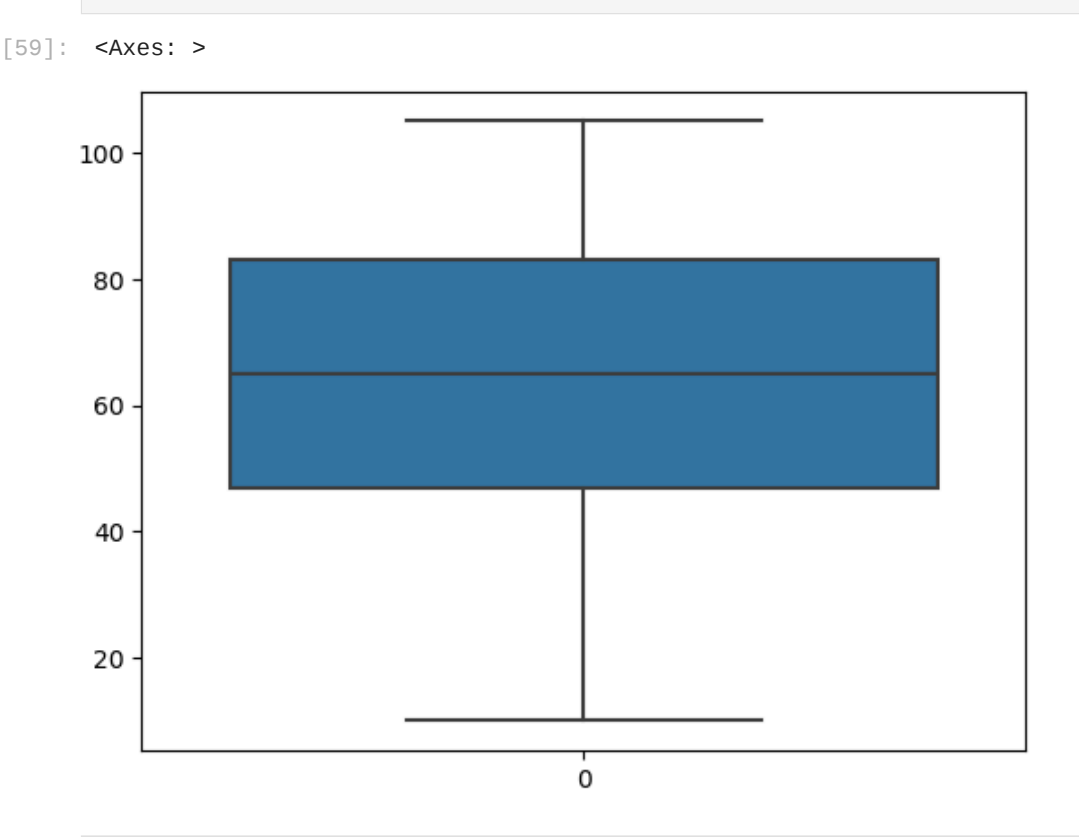
Out[54]:
```
22.860006194736997
```

In [58]:
```python
zscore = (df['tran_amount']-df['tran_amount'].mean())/df['tran_amount'].std()
threshold = 3
outliers = df[abs(zscore)>threshold]
if outliers.empty:
    print('No outliers')
else:
    print('Outliers')
```

```
No outliers
```

In [59]:
```python
sns.boxplot(df.tran_amount)
```

Out[59]:
```
<Axes: >
```



In [60]:
```python
df
```

Out[60]:

	customer_id	trans_date	tran_amount	response
0	CS5295	2013-02-11	35	1.0
1	CS4768	2015-03-15	39	1.0
2	CS2122	2013-02-26	52	0.0
3	CS1217	2011-11-16	99	0.0
4	CS1850	2013-11-20	78	0.0
...	...	...	...	...
124995	CS8433	2011-06-26	64	0.0
124996	CS7232	2014-08-19	38	0.0
124997	CS8731	2014-11-28	42	0.0
124998	CS8133	2013-12-14	13	0.0
124999	CS7996	2014-12-13	36	0.0

125000 rows × 4 columns

In [79]:
```python
customer_counts = df['customer_id'].value_counts().reset_index()
```

In [80]:
```python
customer_counts
```

Out[80]:

	customer_id	count
0	CS4424	39
1	CS4320	38
2	CS3799	36
3	CS1215	35
4	CS5109	35
...	...	...
6884	CS8559	4
6885	CS7716	4
6886	CS8376	4
6887	CS7224	4
6888	CS7333	4

6889 rows × 2 columns

In [82]:
```python
top_5_customer = customer_counts.sort_values( by = 'count', ascending = False).head(5)
```

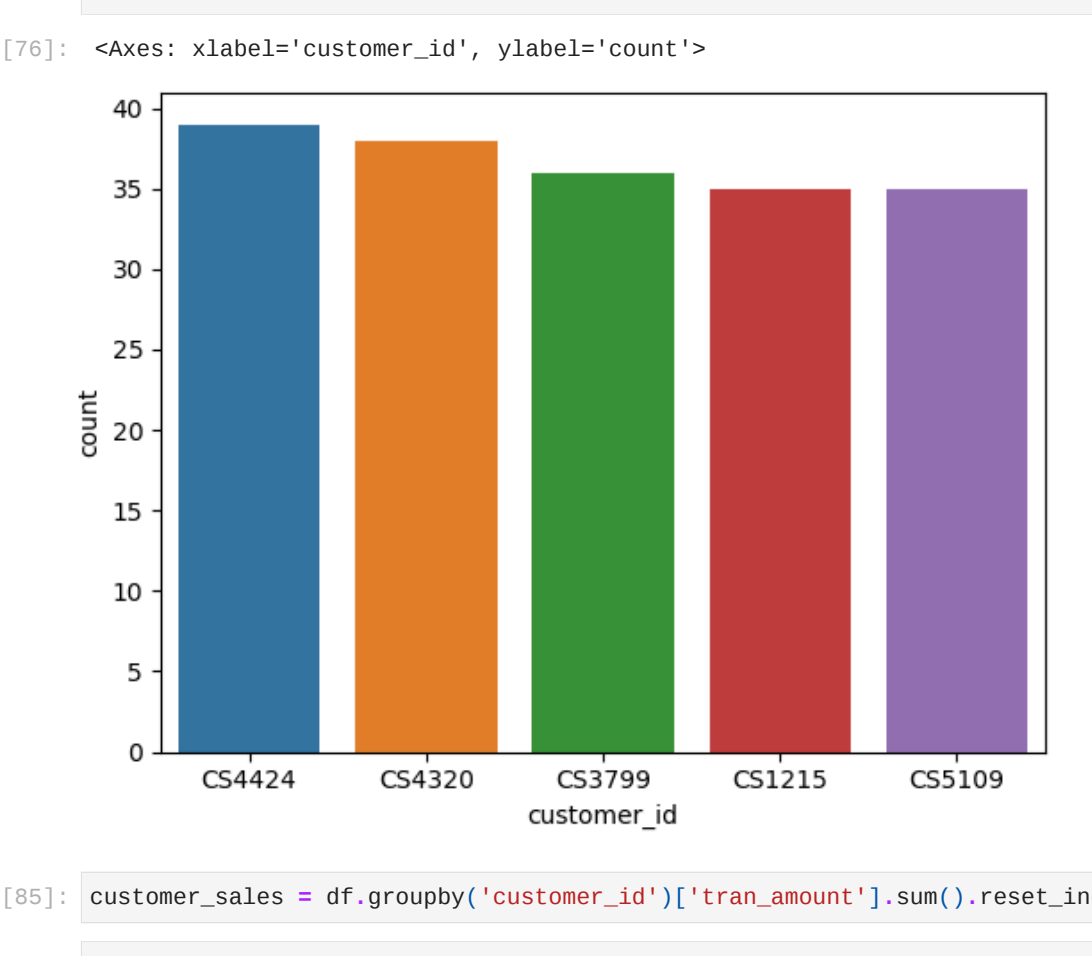
In [74]:
```python
top_5_customer
```

Out[74]:

	customer_id	count
0	CS4424	39
1	CS4320	38
2	CS3799	36
3	CS1215	35
4	CS5109	35

In [76]:
```python
sns.barplot(x = 'customer_id', y = 'count', data = top_5_customer)
```

Out[76]:
```
<Axes: xlabel='customer_id', ylabel='count'>
```



In [85]:
```python
customer_sales = df.groupby('customer_id')['tran_amount'].sum().reset_index()
```

In [86]:
```python
customer_sales
```

Out[86]:

	customer_id	tran_amount
0	CS1112	1012
1	CS1113	1490
2	CS1114	1432
3	CS1115	1659
4	CS1116	857
...	...	...
6884	CS8996	582
6885	CS8997	543
6886	CS8998	624
6887	CS8999	383
6888	CS9000	533

6889 rows × 2 columns

In [87]:
```python
top_5_sales = customer_sales.sort_values(by = 'tran_amount', ascending = False).head(5)
```

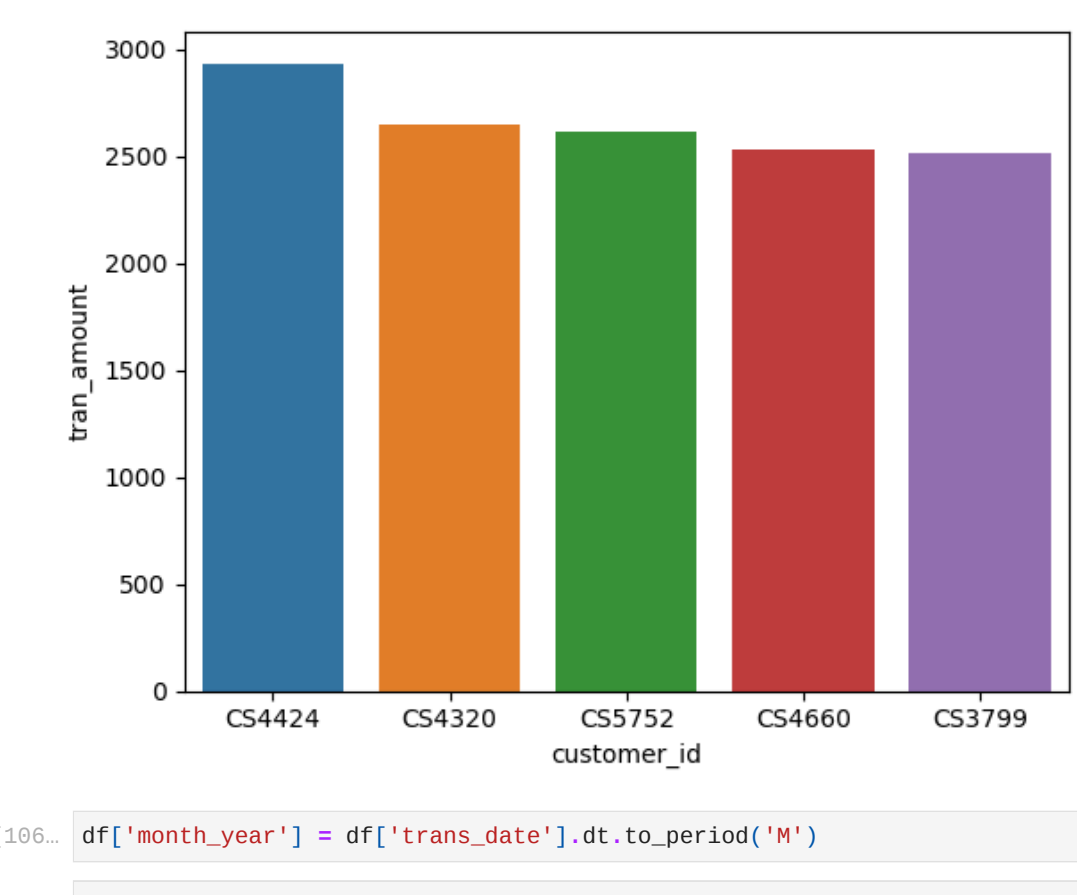
In [88]:
```python
top_5_sales
```

Out[88]:

	customer_id	tran_amount
3312	CS4424	2933
3208	CS4320	2647
4640	CS5752	2612
3548	CS4660	2527
2687	CS3799	2513

In [98]:
```python
sns.barplot(x = 'customer_id', y = 'tran_amount' , data = top_5_sales)
```

Out[98]:
```
<Axes: xlabel='customer_id', ylabel='tran_amount'>
```



In [106…
```python
df['month_year'] = df['trans_date'].dt.to_period('M')
```

In [107…
```python
df
```

Out[107…

	customer_id	trans_date	tran_amount	response	month_year
0	CS5295	2013-02-11	35	1.0	2013-02
1	CS4768	2015-03-15	39	1.0	2015-03
2	CS2122	2013-02-26	52	0.0	2013-02
3	CS1217	2011-11-16	99	0.0	2011-11
4	CS1850	2013-11-20	78	0.0	2013-11
...	...	...	...	...	...
124995	CS8433	2011-06-26	64	0.0	2011-06
124996	CS7232	2014-08-19	38	0.0	2014-08
124997	CS8731	2014-11-28	42	0.0	2014-11
124998	CS8133	2013-12-14	13	0.0	2013-12
124999	CS7996	2014-12-13	36	0.0	2014-12

125000 rows × 5 columns

In [136…
```python
monthly_yearly_10_sales = df.groupby('month_year')['tran_amount'].sum().sort_values(ascending = False).reset_index().head(10)
```

In [137…
```python
monthly_yearly_10_sales
```

Out[137…

	month_year	tran_amount
0	2011-08	188631
1	2013-05	185826
2	2014-01	184554
3	2011-10	182634
4	2012-10	181862
5	2013-01	181729
6	2013-10	181521
7	2013-03	181491
8	2011-12	181405
9	2013-07	180983

In [115…
```python
df['month'] = df['trans_date'].dt.month
```

In [116…
```python
df
```

Out[116…

	customer_id	trans_date	tran_amount	response	month_year	month
0	CS5295	2013-02-11	35	1.0	2013-02	2
1	CS4768	2015-03-15	39	1.0	2015-03	3
2	CS2122	2013-02-26	52	0.0	2013-02	2
3	CS1217	2011-11-16	99	0.0	2011-11	11
4	CS1850	2013-11-20	78	0.0	2013-11	11
...	...	...	...	...	...	...
124995	CS8433	2011-06-26	64	0.0	2011-06	6
124996	CS7232	2014-08-19	38	0.0	2014-08	8
124997	CS8731	2014-11-28	42	0.0	2014-11	11
124998	CS8133	2013-12-14	13	0.0	2013-12	12
124999	CS7996	2014-12-13	36	0.0	2014-12	12

125000 rows × 6 columns

In [117…
```python
monthly_5_sales = df.groupby('month')['tran_amount'].sum().sort_values(ascending = False).head(5).reset_index()
```

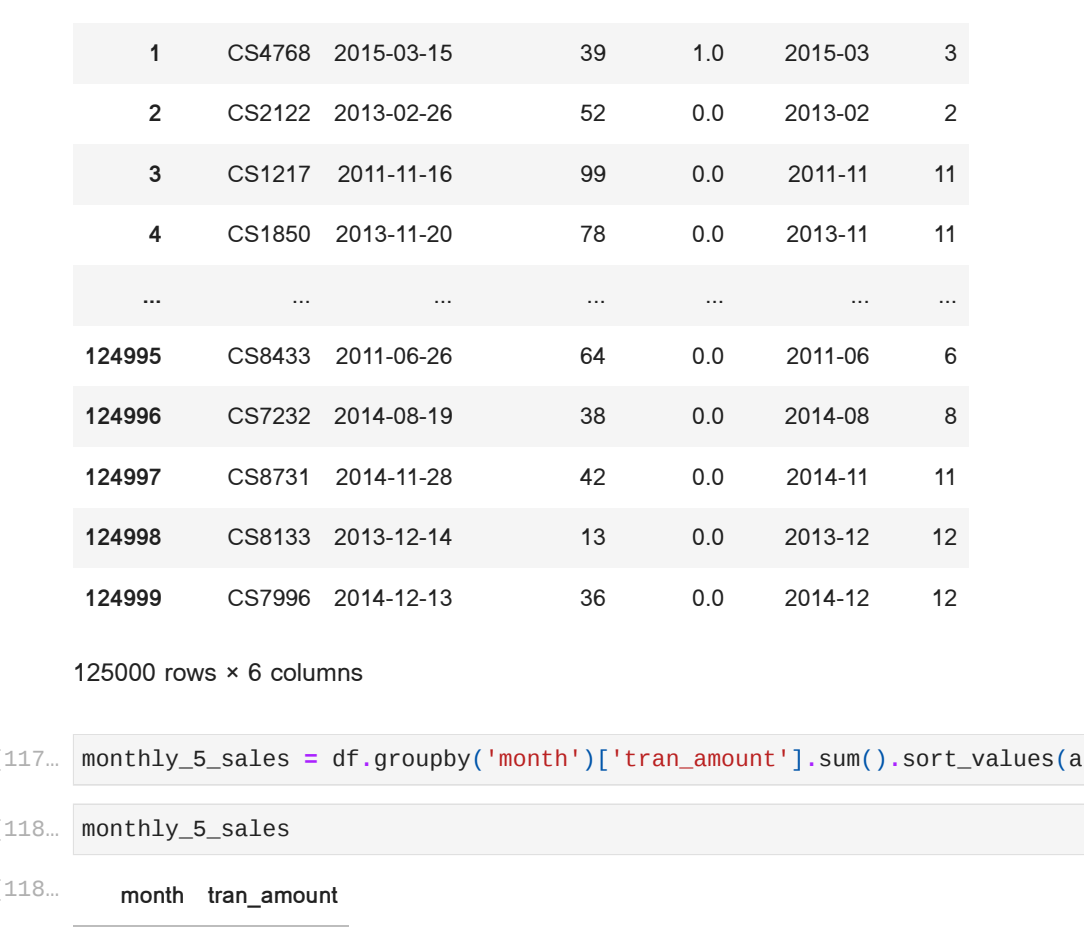
In [118…
```python
monthly_5_sales
```

Out[118…

	month	tran_amount
0	8	726921
1	10	725320
2	1	724107
3	7	717223
4	12	709964

In [125…
```python
sns.regplot( x = 'month', y = 'tran_amount', data = monthly_5_sales)
```

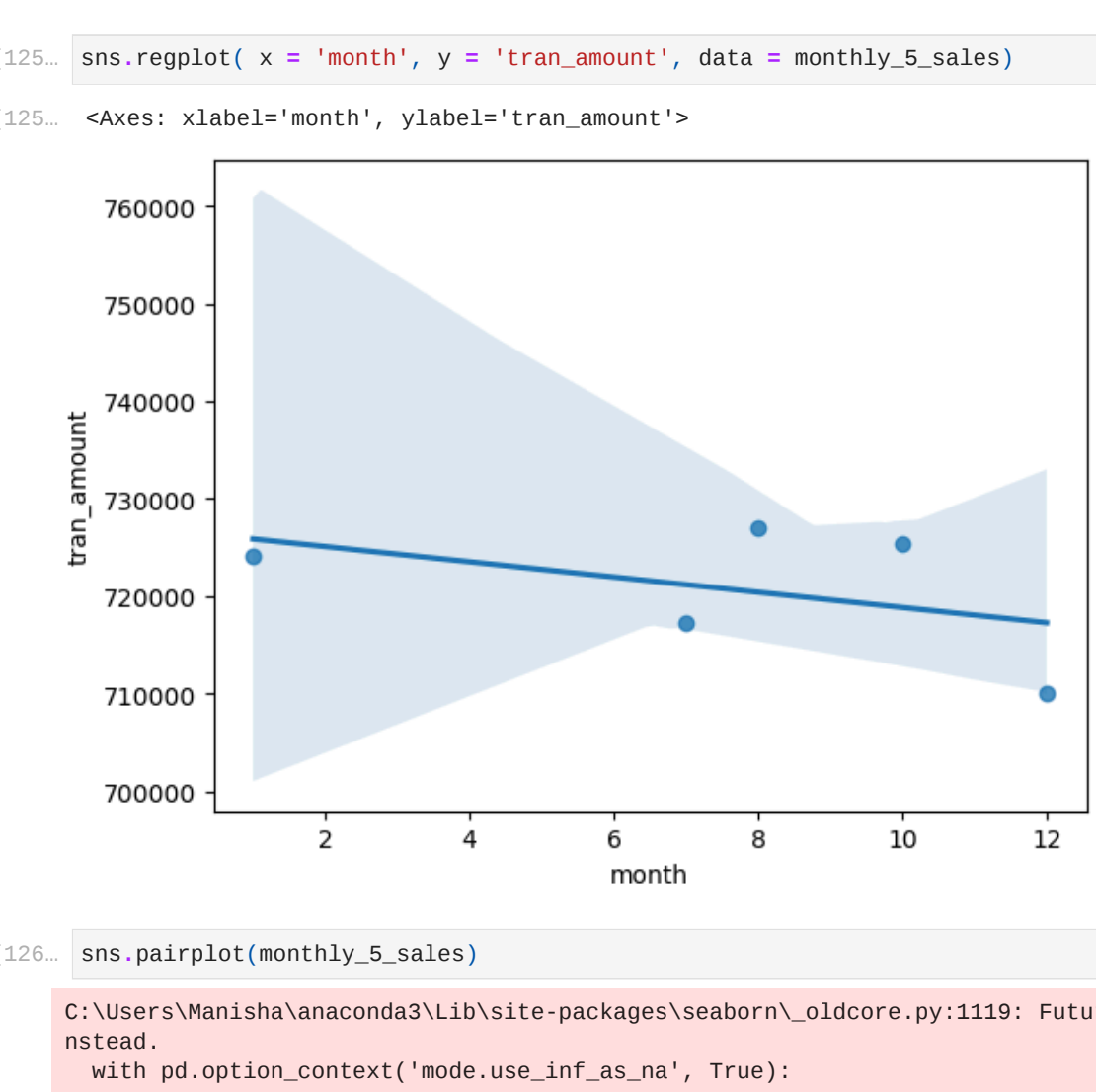
Out[125…
```
<Axes: xlabel='month', ylabel='tran_amount'>
```



In [126…
```python
sns.pairplot(monthly_5_sales)
```

```
C:\Users\Manisha\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating i
nstead.
  with pd.option_context('mode.use_inf_as_na', True):
C:\Users\Manisha\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating i
nstead.
  with pd.option_context('mode.use_inf_as_na', True):
```

Out[126…
```
<seaborn.axisgrid.PairGrid at 0x1a88da124d0>
```

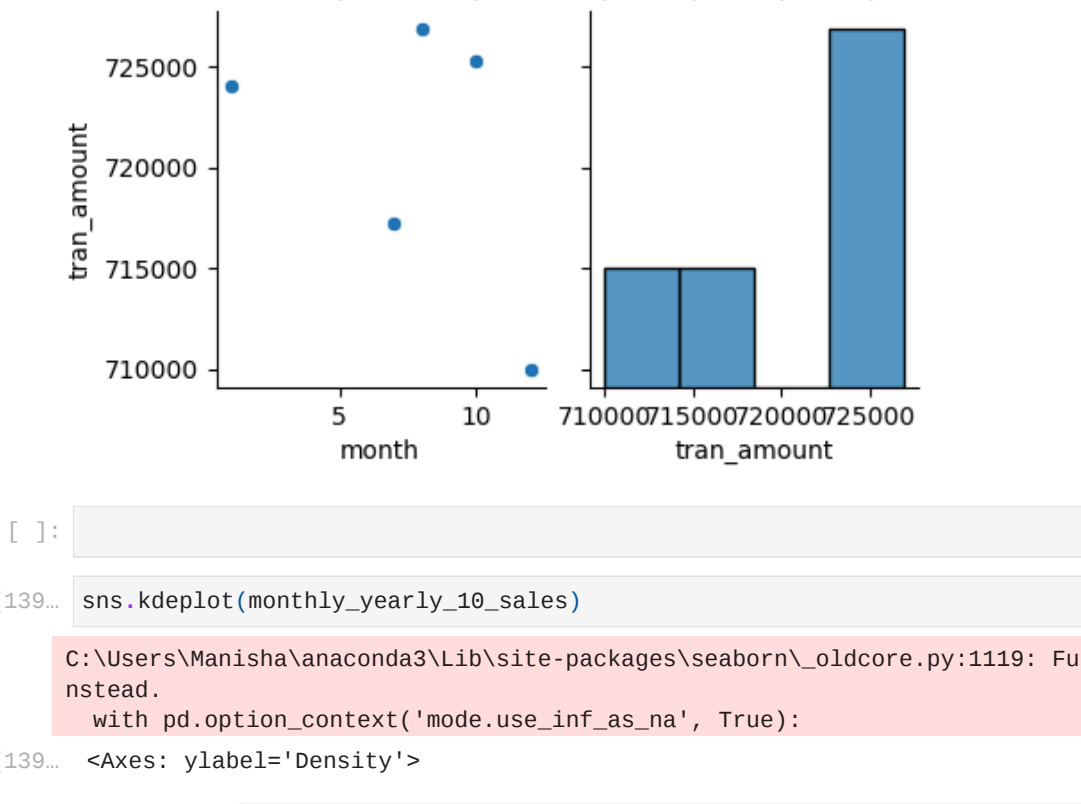


In [ ]:

In [139…
```python
sns.kdeplot(monthly_yearly_10_sales)
```

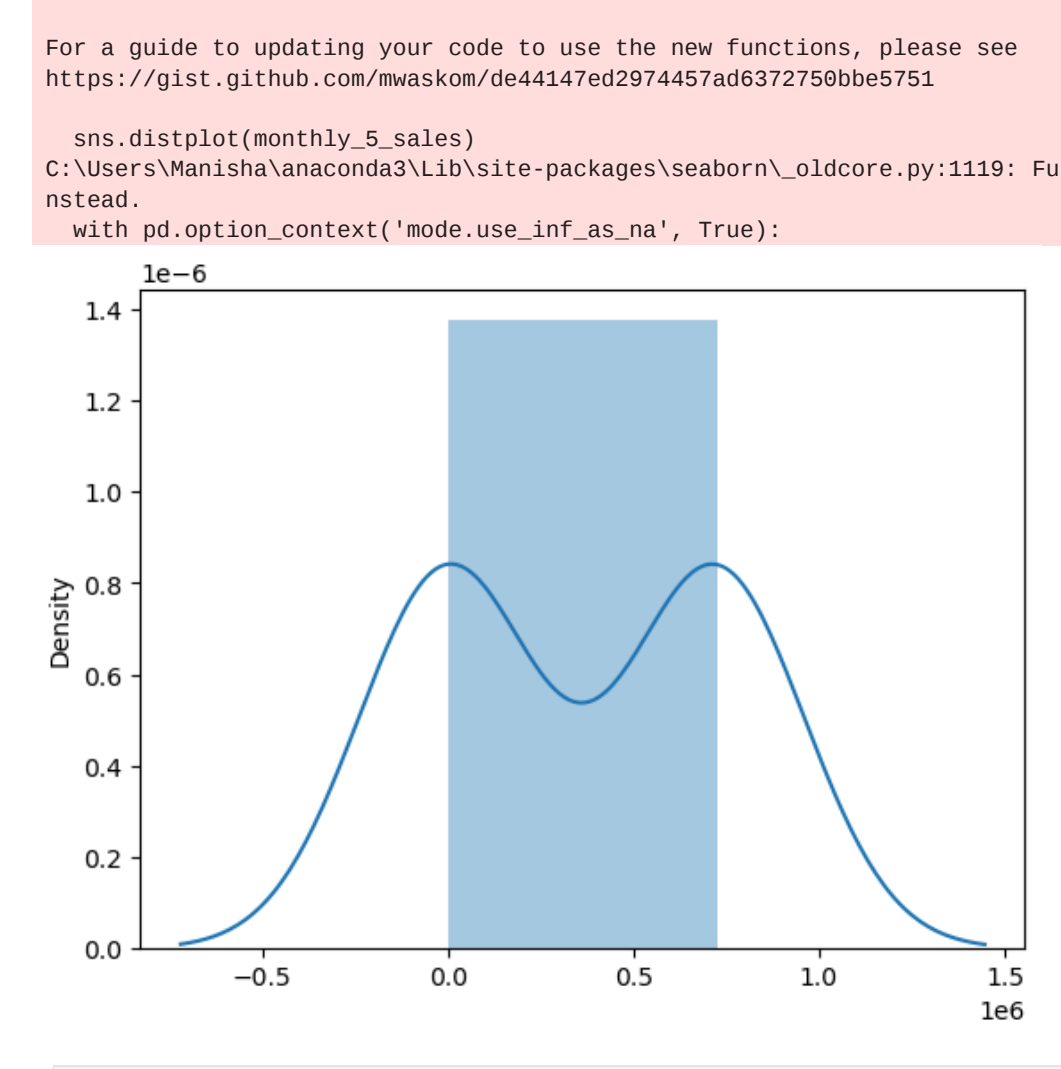
```
C:\Users\Manisha\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating i
nstead.
  with pd.option_context('mode.use_inf_as_na', True):
```

Out[139…
```
<Axes: ylabel='Density'>
```



In [143…
```python
sns.distplot(monthly_5_sales)
plt.show()
```

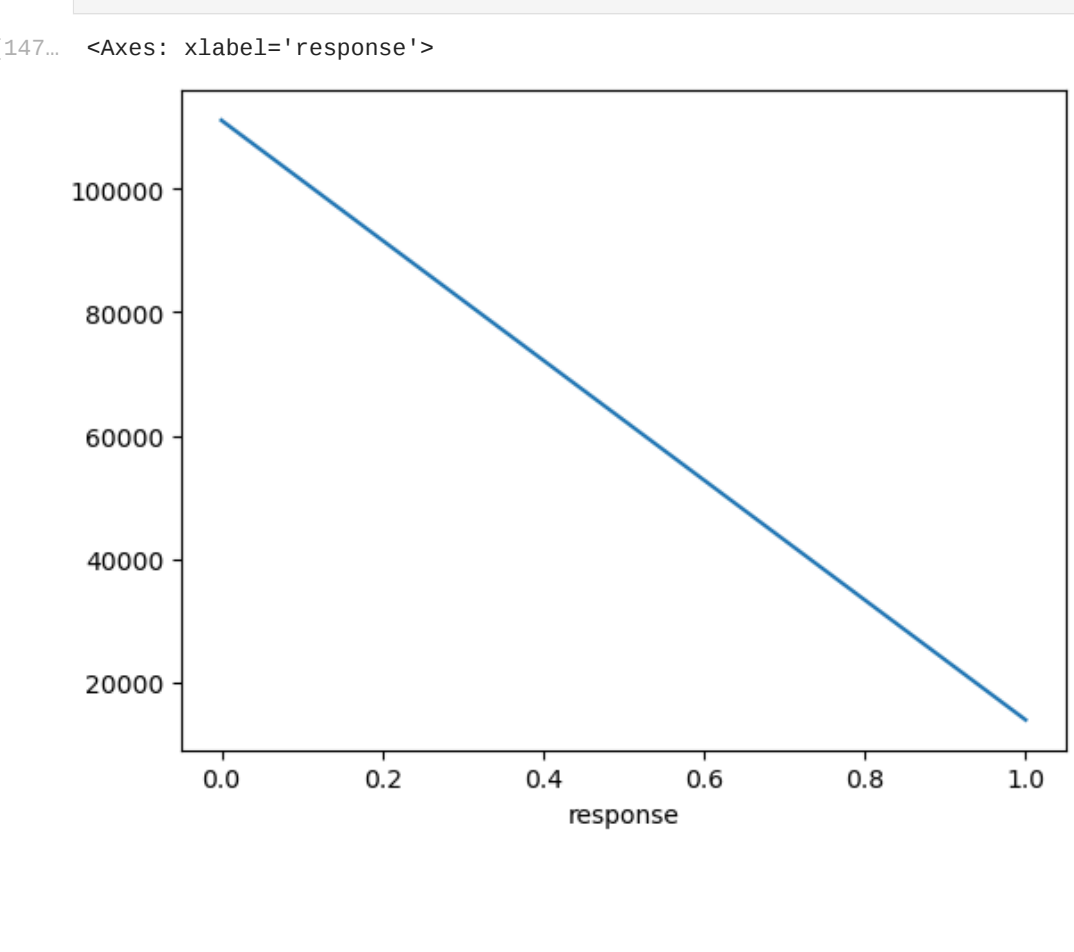
```
C:\Users\Manisha\AppData\Local\Temp\ipykernel_10324\542403786.py:1: UserWarning: 

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(monthly_5_sales)
C:\Users\Manisha\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating i
nstead.
  with pd.option_context('mode.use_inf_as_na', True):
```



In [144…
```python
churn_analysis = df['response'].value_counts()
```

In [147…
```python
churn_analysis.plot(kind = 'line')
```

Out[147…
```
<Axes: xlabel='response'>
```



```
In [ ]:
```

```
In [ ]:
```